DP Computer Science: B 2.2.4 Queues (Python)

---

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Understand the **FIFO principle** and how queues operate
- Implement queue operations (**enqueue, dequeue, front, isEmpty**) in Python
- Analyse the **performance and memory usage** of queue operations
- Choose appropriate **queue implementations** for different scenarios in Python
- Apply **queues to solve problems** (e.g., message queue, simulation)

Key Vocabulary

Term	Definition
Queue	A data structure that follows the First In, First Out (FIFO) principle
FIFO	First In, First Out — the first item added is the first to be removed
Enqueue	Adding an item to the back of the queue
Dequeue	Removing and returning the item from the front of the queue
Front	Viewing the item at the front of the queue without removing it
Rear/Back	The position at the back of the queue where new items are added
isEmpty	Checking whether the queue is empty

IB ATL Skills Focus

Skill	Description
Thinking	Critical thinking (analysing queue behaviour); creative thinking (solving problems with queues)
Research	Information literacy (researching different queue implementations)
Communication	Collaboration (pair programming); explaining queue concepts
Self-Management	Organisation (managing code); time management (completing tasks efficiently)

---

2. Introduction to Queues

The Real-World Analogy

> *"A queue is like a line at a store. People join the back of the line, and the person at the front is served first. The first person in line is the first person to leave."*

The FIFO Principle

```
text

┌──────────────────────────────────────────────────────────────┐
│              QUEUE FIFO PRINCIPLE                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│    ┌──────────────────────────────────────────────────┐      │
│    │  Front →   [10]   [20]   [30]   [40]   [50]  ← Rear  │  │
│    │             ↑                          ↑         │  │
│    │         Dequeue                    Enqueue       │  │
│    │         (Remove)                   (Add)         │  │
│    └──────────────────────────────────────────────────┘      │
│                                                              │
│    First In, First Out (FIFO)                                │
│    The first item added is the first to be removed           │
```

Real-World Examples

Example	How a Queue Is Used
Checkout Line	Customers join the back and are served from the front
Printer Queue	Print jobs are processed in the order they are received
Message Queue	Messages are delivered in the order they are sent
Web Server Requests	Requests are handled in the order they arrive
Call Centre	Calls are answered in order of arrival

---

3. Queue Operations

Core Operations

Operation	Description	Python Implementation
enqueue(item)	Adds an item to the back of the queue	`list.append(item)`
dequeue()	Removes and returns the item from the front	`list.pop(0)`
front()	Returns the item at the front without removing	`list[0]`
isEmpty()	Returns True if the queue is empty	`len(list) == 0`

Using Python Lists as Queues

```python
# Creating a queue using a list
queue = []

# Enqueue operations
queue.append(10)  # Add to back
queue.append(20)
queue.append(30)
print(queue)  # Output: [10, 20, 30]

# Dequeue operation
front_item = queue.pop(0)  # Remove from front
print(front_item)  # Output: 10
print(queue)  # Output: [20, 30]

# Front operation
front = queue[0]
print(front)  # Output: 20
print(queue)  # Output: [20, 30]  (unchanged)

# Check if empty
if len(queue) == 0:
    print("Queue is empty")
else:
    print(f"Queue has {len(queue)} items")
```

Performance Consideration

Operation	List Implementation	Time Complexity
Enqueue	`list.append()`	O(1)
Dequeue	`list.pop(0)`	O(n) — shifts all elements
Front	`list[0]`	O(1)

> **Important:** Using `pop(0)` on a list is O(n) because all remaining elements must be shifted. For better performance, use `collections.deque`.

4. Implementing a Queue

Method 1: Using a List (Simple but Inefficient)

python

```python
class Queue:
    def __init__(self):
        """Initialize an empty queue."""
        self.items = []

    def is_empty(self):
        """Check if the queue is empty."""
        return len(self.items) == 0

    def enqueue(self, item):
        """Add an item to the back of the queue."""
        self.items.append(item)
        print(f"Enqueued: {item}")

    def dequeue(self):
        """Remove and return the item from the front of the queue."""
        if self.is_empty():
            print("Error: Queue is empty.")
            return None
        item = self.items.pop(0)
        print(f"Dequeued: {item}")
        return item

    def front(self):
        """Return the item at the front without removing it."""
        if self.is_empty():
            print("Error: Queue is empty.")
            return None
        return self.items[0]

    def size(self):
        """Return the number of items in the queue."""
        return len(self.items)

    def display(self):
        """Display all items in the queue."""
        print(f"Queue: {self.items}")
```

```
# Example usage
q = Queue()
q.enqueue(5)
q.enqueue(10)
q.enqueue(15)
q.display()          # Queue: [5, 10, 15]
print(q.front())     # 5
q.dequeue()          # Removes 5
q.display()          # Queue: [10, 15]
print(q.size())      # 2
```

Method 2: Using collections.deque (Efficient)

python

```
from collections import deque

class EfficientQueue:
    def __init__(self):
        """Initialize an empty queue using deque."""
        self.items = deque()

    def is_empty(self):
        """Check if the queue is empty."""
        return len(self.items) == 0

    def enqueue(self, item):
        """Add an item to the back of the queue (O(1))."""
        self.items.append(item)
        print(f"Enqueued: {item}")

    def dequeue(self):
        """Remove and return the item from the front (O(1))."""
        if self.is_empty():
            print("Error: Queue is empty.")
            return None
        item = self.items.popleft()
        print(f"Dequeued: {item}")
        return item

    def front(self):
```

```
        """Return the item at the front without removing it (O(1))."""
        if self.is_empty():
            print("Error: Queue is empty.")
            return None
        return self.items[0]

    def size(self):
        """Return the number of items in the queue (O(1))."""
        return len(self.items)

    def display(self):
        """Display all items in the queue."""
        print(f"Queue: {list(self.items)}")

# Example usage
q = EfficientQueue()
q.enqueue(5)
q.enqueue(10)
q.enqueue(15)
q.display()          # Queue: [5, 10, 15]
print(q.front())     # 5
q.dequeue()          # Removes 5
q.display()          # Queue: [10, 15]
```

---

5. Queue Implementations Comparison

Python List vs. deque

Feature	List	deque
Enqueue	O(1)	O(1)
Dequeue	O(n) — shifts elements	O(1)
Front	O(1)	O(1)
Memory	May overallocate	Efficient

Feature	List	deque
Use When	Simple use cases	Performance-critical applications

Available Queue Implementations

Implementation	Description
list	Simple but inefficient for dequeue
collections.deque	Fast O(1) operations on both ends
queue.Queue	Thread-safe implementation for multithreading
Custom LinkedList	No resizing overhead

---

6. Queue Applications

Application 1: Printer Queue Simulation

```python
class PrintJob:
    def __init__(self, doc_name, pages):
        self.doc_name = doc_name
        self.pages = pages

    def __str__(self):
        return f"{self.doc_name} ({self.pages} pages)"

class Printer:
    def __init__(self):
        self.queue = []

    def add_job(self, job):
        self.queue.append(job)
        print(f"Added print job: {job}")
```

```
    def process_job(self):
        if not self.queue:
            print("No print jobs in queue.")
            return None
        job = self.queue.pop(0)
        print(f"Printing: {job}")
        return job

    def display_queue(self):
        print("Print Queue:")
        for i, job in enumerate(self.queue):
            print(f"  {i+1}. {job}")

# Example usage
printer = Printer()
printer.add_job(PrintJob("Report.pdf", 5))
printer.add_job(PrintJob("Image.png", 2))
printer.add_job(PrintJob("Document.docx", 10))
printer.display_queue()
printer.process_job()
printer.process_job()
```

Application 2: Web Server Request Queue

python

```
class Request:
    def __init__(self, client, path):
        self.client = client
        self.path = path

    def __str__(self):
        return f"Request from {self.client} for {self.path}"

class WebServer:
    def __init__(self):
        self.queue = []

    def add_request(self, request):
        self.queue.append(request)
        print(f"New request received: {request}")
```

```
    def process_request(self):
        if not self.queue:
            print("No requests in queue.")
            return None
        request = self.queue.pop(0)
        print(f"Processing: {request}")
        return request

    def queue_status(self):
        print(f"{len(self.queue)} requests waiting.")

# Example usage
server = WebServer()
server.add_request(Request("192.168.1.1", "/home"))
server.add_request(Request("192.168.1.2", "/about"))
server.add_request(Request("192.168.1.3", "/contact"))
server.queue_status()
server.process_request()
server.process_request()
```

Application 3: Breadth-First Search (BFS)

python

```
from collections import deque

def bfs(graph, start):
    """Perform Breadth-First Search using a queue."""
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        vertex = queue.popleft()
        print(vertex, end=" ")

        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

# Example graph
```

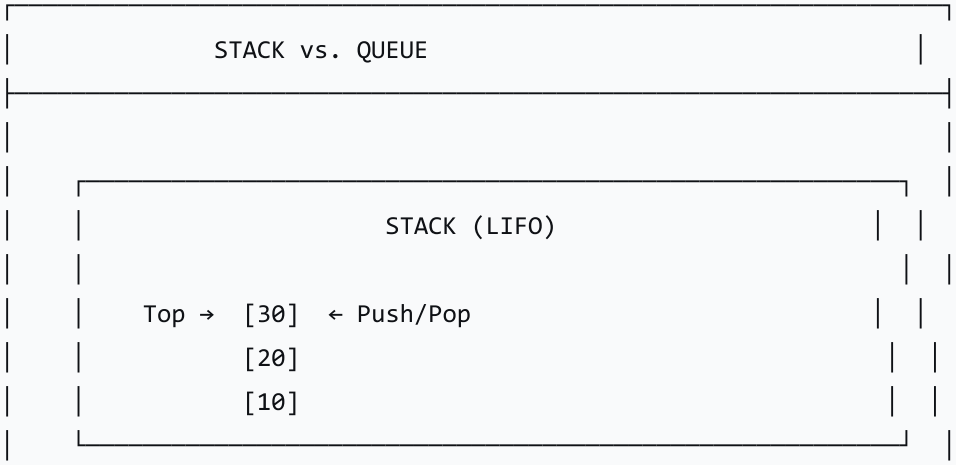
```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B'],
    'F': ['C']
}

print("BFS traversal starting from A:")
bfs(graph, 'A')  # Output: A B C D E F
```

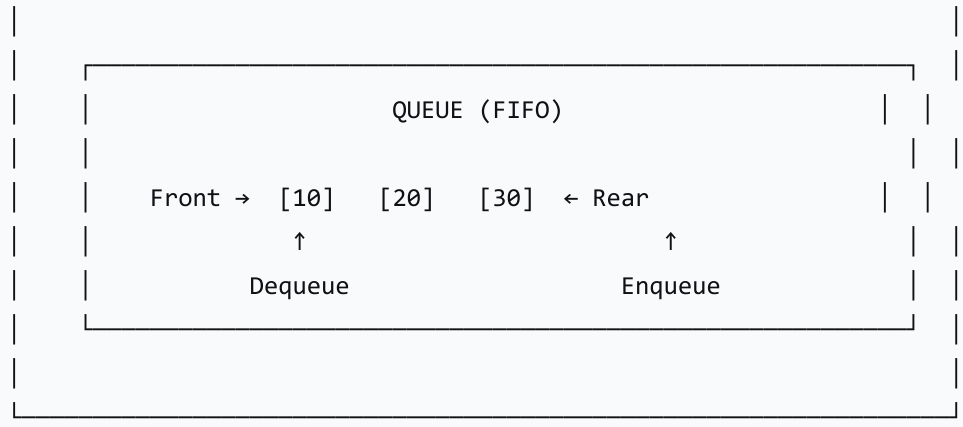
---

7. Queue vs. Stack Comparison

Feature	Stack (LIFO)	Queue (FIFO)
Principle	Last In, First Out	First In, First Out
Add	Push (to top)	Enqueue (to back)
Remove	Pop (from top)	Dequeue (from front)
View	Peek (top)	Front
Use Cases	Undo, back button, recursion	Print queue, request queue

```
text
```





---

8. Worksheet Practice

Task 1: Basic Queue Operations

```python

# 1. Create an empty queue
# 2. Enqueue the numbers 5, 10, 15, 20
# 3. Dequeue and print the front element
# 4. View the front element
# 5. Check if the queue is empty
# 6. Enqueue the number 25
# 7. Dequeue all elements and print them
```

Task 2: Simulate a Customer Service Queue

```python

class Customer:
    def __init__(self, name, service_time):
        self.name = name
        self.service_time = service_time

# Create a queue of customers
```

```
    # Process each customer in order
    # Track total time
```

Task 3: Implement a Circular Queue

```python
class CircularQueue:
    def __init__(self, capacity):
        # Your code here
        pass

    def enqueue(self, item):
        # Your code here
        pass

    def dequeue(self):
        # Your code here
        pass

    def is_full(self):
        # Your code here
        pass
```

---

9. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Introduction	10 min	Queue analogy (line at a store); FIFO principle
Queue Operations	15 min	Enqueue, dequeue, front, isEmpty; Python implementations

Phase	Time	Activity
Performance	5 min	Time complexity; list vs. deque
Implementations	15 min	Students implement Queue class
Applications	10 min	Real-world applications; code examples
Wrap-up	5 min	Review; worksheet practice

10. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Implementation Activity	Assess ability to implement queue operations
Worksheet	Evaluate understanding of queue applications

11. Differentiation

Level	Strategy
Support	Provide scaffolded code examples
Challenge	Implement circular queue; explore concurrent queues

12. Resources

- **Presentation** (Slide Deck)
- **eBook** (B2.2.4 with examples)
- **Worksheet:** Queue operations and applications

- Worksheet Answers

---

13. Summary: Key Takeaways

Concept	Key Point
Queue	FIFO data structure
Enqueue	Add to the back
Dequeue	Remove from the front
Front	View the front
FIFO	First In, First Out
deque	Efficient Python queue implementation

```
text

┌──────────────────────────────────────────────────────────────┐
│                    KEY REMINDERS                             │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│    ✅ Queue = FIFO data structure                           │
│    ✅ Enqueue = add item to back                            │
│    ✅ Dequeue = remove item from front                      │
│    ✅ Front = view front item without removing              │
│    ✅ Use collections.deque for efficient queues            │
│    ✅ Real-world uses: print queues, request queues, BFS    │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

> *"Queues are essential for managing tasks in order—they ensure that the first item added is the first to be processed."*